Software Requirement Specification, Version 1.2

Prepared on 2026-05-17

For the American Society of Cybernetic's Collaborative Knowledge Hub. By Group 2 of Curtin's Capstone Computing Project 1.

Previous version: 📄 2026-04-28 - CKH SRS 1.1.pdf

Changelog

- Added Appendix B: Handover Plan
- Added Appendix C: UI
- Per 📅 2026-04-30 Client Meeting:
    - Updated the format for extensions. It is now more granular: per-module rather than having system-wide extensions.
- Updates to 5.2 Safety and 5.3 Security
- Updates to functionality in 3.1 Member Management; added the concept of membership applications.
- Made 3.2 Groups more specific - including automatically syncing with the connected Google Workspace.
- Minor edits to 3.3 Knowledge Hub: Created an "Automated Summary" extension. Added the ability to make approval optional for a repository.
- Reprioritisation of 3.5 Event Creation and Coordination
- Added a basic implementation of 3.6 Knowledge Graph that is in the MVP and elaborate on specific techniques we intend to use.
- Added a basic implementation of 3.7 Member Matching that is in the MVP
- Added a basic implementation of 3.8 Automated Newsletter. Gave specific examples of content that should be in the newsletter.
- Update the functionality of 3.9 Research Harvester to give a more precise definition of the scraping. Moved the concept of a "Feed" to a lower priority extension.
- Rework 3.10 Automated Metadata Suggestion. It now includes tags for member profiles.
- Added 3.11 User Journeys
- Added 3.12 Feedback

Table of Contents

---

1. Intro

1.1 Project Description

The Collaborative Knowledge Hub (CKH) is envisioned as a new, standalone responsive web app that integrates multiple systems traditionally found across separate platforms — membership management, event coordination, research discovery, and collaborative knowledge creation — into a single cohesive ecosystem. It will act as both a community portal and a knowledge infrastructure for the American Society for Cybernetics (ASC).

The platform must support complex, interconnected data structures, including relationships between resources such as members, member-produced articles, external publications, glossary terms, and events. Manual and automated metadata tagging will serve as the backbone for producing a knowledge graph, in order to produce visualisations of these relationships and enable intuitive navigation.

The system also incorporates AI-powered features, such as automated content synthesis, smart recommendations, and external resource harvesting, making it more than a traditional membership portal. It is designed to evolve over time as members contribute content, annotate resources, and engage in collaborative activities.

1.2 Definitions

CKH: The Collaborative Knowledge Hub (the product this document describes).

Extension Priority X: sections with this tag aren't in the MVP. See Appendix A: Extension Priority for what extensions exist at each priority level.

1.2.1 User Roles

See 2.3 User Characteristics for more detail.

- Member: an ASC member.
- Moderator: a member with some permissions for a particular module, usually for moderating content on that module. These permissions are per-module, and some modules may define more granular permissions (e.g. someone can be a moderator of all events, or just some specific events).
- Manager: a member with permissions for a module, and some modules may define more granular permissions (e.g. someone can be the manager of all events, or just some specific events).
- Administrator: a member with permission to configure the system.

---

2. Overall Description

2.1 Product Perspective

The ASC Knowledge Hub is a standalone web-based platform designed for the American Society for Cybernetics (ASC). Its purpose is to centralise ASC publications, glossary terms, member-generated resources, and cybernetics-related knowledge into a single, searchable, and extensible system.

The system follows a three-tier architecture consisting of:

- Front-end client: A reactive, browser-based interface responsible for browsing, searching, editing, and interacting with ASC resources.
- middleware/API layer handles business logic, authentication, authorisation, OTP-based security, and communication between the client and database.
- Database layer — storing structured metadata, glossary terms, publications, user accounts, and relationships between concepts.

This architecture supports modularity, long-term extensibility, and the addition of new knowledge modules as ASC evolves. The knowledge Hub will be accessible through modern web browsers on both desktop and mobile devices.

2.2 Product Functions

The platform will include a wide range of high-level capabilities:

Member management

- Profile creation with fields for research interests, affiliations, publications, and personal statements.
- Membership status tracking, including renewal reminders and fee processing.
- Secure authentication and role-based access control.

Collaborative knowledge creation

- A living glossary of cybernetic terms curated by members.
- A shared publication and resource library with annotation and linking features.
- Version control for knowledge hub entries.

[Extension] Event creation and coordination

- Tools for creating online or in-person gatherings.
- Event pages with RSVP tracking and reminders.
- Integrated knowledge creation functionalities for content (e.g. abstracts) submission and review.

Knowledge graph visualization

- Interactive graphs showing relationships between resources such as members, member-written articles, external publications, and events.
- Multiple visualization modes to accommodate different cognitive styles.
- Filters, clustering and semantic tagging for deeper exploration.

[Extension] AI-powered discovery and content synthesis

- AI tools that search online for relevant papers.

Automated Monthly Newsletter

- Automated monthly summaries highlight trends, new members, active discussions, and emerging research.

Smart member matching

- Algorithms that analyze member profiles, interests, and activity patterns.
- Suggestions for potential collaborators, mentors, or project partners.
- Dynamic matching that updates as members contribute new content.

Communal feedback on knowledge hub content

- Rating, commenting, bookmarking and "mark as read" systems for knowledge hub content.
- Upvotes, downvotes, and flagging for communal moderation.
- [Extension] AI-generated summary of comments.

2.3 User Characteristics

The system will be used by:

- General public users interested in cybernetics
    - Individuals interested in cybernetics, system theory, or ASC publications.
    - Varying levels of digital literacy.
    - Primarily consumers of content rather than contributors.
- ASC members (academics, researchers, practitioners)
    - Typically familiar with the academic platforms, digital libraries, and research tools.
    - May contribute new resources, glossary terms, or publications.
    - Require reliable search, citation-friendly metadata, and stable access to archival materials.
    - Expected to use the platform on both desktop and [Extension] mobile devices.
- Moderators
    - Moderates a specific module, or something more granular as defined per-module.
    - Responsible for content moderation, metadata curation, and maintenance.
- Managers
    - Manages a specific module, or something more granular as defined per-module.
    - Responsible for event creation, payment review, publication updates, and membership management.
    - Expected to perform tasks such as:
        - Performing data imports and exports for a module
        - Members Manager — Managing user roles
        - Payments Manager — Auditing payments
        - Memberships Manager — Membership statuses
        - Events Manager — Creating events
- Administrators
    - Require elevated permissions and access to administrative dashboards.
    - Expected to perform tasks such as:
        - Configuring modules
        - Performing data imports and exports
        - Managing user roles
    - May require onboarding to understand metadata structures and editorial workflows.

Users are expected to have general digital literacy but do not require technical expertise. Administrators may require brief onboarding to use the content management features.

2.4 Operating Environment

The ASC Knowledge Hub operates within a dedicated server, possibly a Virtual Private Server (VPS). The system is designed to be cost-efficient for the ASC, a non-profit organisation, while still supporting modern web application standards, containerised deployment, secure authentication, and long-term maintainability. This section describes the full technical environment required for the system to function, including client-side, server-side, database, hosting, deployment, network, hardware, and AI-related requirements.

Note that technologies listed in this section are not final: this is our intended stack, and so influence the operating environment, but elements may change during development.

2.4.1 Hardware Environment

Client-side hardware

- The system is designed to run on standard consumer devices with no specialised hardware requirements:
    - Desktop or laptop computer
    - Smartphone or tablets
    - Minimum 4 GB RAM recommended for smooth browser performance
    - Minimum display resolution: 1280×720
    - Recommended resolution: 1920×1080
    - Touchscreen support for mobile and tablet users

Server-side hardware

- The ASC Knowledge Hub is deployed on a standalone Linux server. This hosting model was selected because it is significantly more cost-effective for the ASC, a non-profit organisation.
    - Minimum Server specification:
        - 2 vCPUs
        - 4 GB RAM
        - 40 GB SSD storage
    - Recommended product specification:
        - 4 vCPUs
        - 8 GB RAM
        - 80+ GB SSD
        - SSD storage for fast I/O
        - off-server backup storage
- The server must support containerised workloads and persistent storage for PostgreSQL.

2.4.2 Software Environment

Client-side software

- React-generated javascript, css and html
- Client-side requirements:
    - Modern browser with ES6 support
    - JavaScript enabled
    - Cookies enabled for session handling
    - CSS3 support for responsive layouts

Server-side software

- Operating system: Linux
- FastAPI server
    - FastAPI
    - Asynchronous request handling
    - Pydantic for schema validation
    - Dependency injection for modularity
- Next.js server
    - React
    - Selected for:
        - Component-based architecture
        - Strong ecosystem and long-term maintainability
        - Excellent documentation and community support
        - Compatibility with REST/JSON APIs
        - High performance through virtual DOM rendering
    - Supports responsive UI design for desktop and mobile devices

        - Integrates with modern build tools
- OJS Application
- Containerisation
    - Docker Compose for orchestrating multi-service deployments
    - Podman as a rootless alternative if preferred or, for some reason, required
- Reverse Proxy
    - Nginx for:
        - HTTPS termination
        - TLS certificate management
        - Routing to Next.js, FastAPI or OJS server
        - Static file delivery
        - Rate limiting

Database

- PostgreSQL is used as the primary storage technology
- Chosen due to its strong relational modelling, JSONB support, ACID compliance, and sustainability for metadata-rich systems
- Persistent volumes for data durability

Authentication and Security libraries

- Argon2-cffi for secure password hashing
- PyOTP for optional two-factor authentication
- JWT/session-based authentication
- Strict CORS and input validation
- Role-based access control

Backend development tools

- pytest
- uv
- Ruff
- MyPy
- Testing responsibilities include:
    - AI endpoints
    - Database operations
    - Authentication flows
    - Error handling

Front-End development tools

- React testing library (RTL) for behaviour-driven component testing
- Jest for unit tests, snapshot tests, and mocking
- Testing responsibilities include:
    - Component rendering
    - User interaction simulation
    - API call mocking
    - Regression testing for UI behaviour

2.4.3 Network Environment

Connectivity Requirements

- Stable internet connection required for all users
- Low latency recommended for administrative tasks

Protocols

- HTTPS required for all communication
- TLS 1.2+ encryption
- REST/JSON API communication

API Security

- Rate limiting via Nginx
- CORS restricted to ASC domains
- Input sanitisation at API layer

Internal networking

- Docker internal network exposes:
    - FastAPI backend
    - PostgreSQL database
    - Next.js front end
    - OJS application
    - Reverse proxy
- The database is not exposed publicly.

2.4.4 Hosting Environment

Hosting characteristics

- Single Server instance
- No auto-scaling or cloud orchestration
- Full control over environment configuration
- SSH access restricted to authorised administrators

Deployment model

- Containerised using Docker for consistent development and deployment
- Services include:
    - FastAPI server (run with the default server, uvicorn)
    - OJS application
    - Next.js server
    - PostgreSQL database
    - Nginx reverse proxy
- Secrets stored outside version control

Backup strategy

- Snapshot-based backups
    - The server filesystem and PostgreSQL data volumes are periodically captured using snapshot-based backups
    - Snapshots provide point-in-time recovery with minimal downtime
    - Snapshots are stored in external object storage (e.g., S3-compatible storage or low-cost object storage provider)
    - Object storage is chosen because it is:
        - Highly durable
        - Extremely cost-effective
        - Independent of the Server

        - Suitable for long-term archival
- Off-Server storage requirement
    - All backups must be stored outside the server to avoid single-point-of-failure scenarios
    - Object storage ensures that backups remain available even if the server becomes corrupted or inaccessible
- Recovery procedures
    - Snapshots restoration for full-system recovery
    - Logical backup restoration for partial recovery

2.4.5 AI Integration environment

Default AI

- Claude used for:
    - Content summarisation
    - Metadata suggestions
    - Internal editorial tools
- Code will be structured to allow easy transition to other models, but they will not be supported

AI Hosting

- All AI processing occurs in external APIs
- No local model hosting required
- No GPU or specialised hardware required

2.4.6 External Integrations

Peer-review systems

- Integration with OJS (Open Journal Systems)
- Supports:
    - Submission workflows
    - Reviewer assignments
    - Metadata synchronisation

Payment platform

- Stripe for:
    - Membership payments
    - Donations
    - Event fees
- PCI compliance handled by Stripe

2.5 Design and Implementation Constraints

The ASC Knowledge Hub must operate within several technical, organisational, and environmental constraints that influence system design, implementation, and deployment.

Note that technologies listed in this section are not final; this is our intended stack, and so create relevant constraints, but elements may change during development.

2.5.1 Technical constraints

Backend framework constraints

- The backend will be implemented using FastAPI, which enforces asynchronous request handling, Pydantic-based validation, and ASGI-compatible deployment.
- The back end will run on an ASGI server, which affects concurrency models and middleware compatibility.

Front-end framework constraints

- The front-end will be built using React, requiring a component-based architecture, JSX syntax, and a modern JavaScript toolchain.
- UI testing will use React Testing Library (RTL) and/or Jest, influencing how components are structured and how side-effects are handled.

Database constraints

- The system will use PostgreSQL as the primary database.
- Data modeling must thus be defined with relational structures and JSONB fields.
- PostgreSQL will run inside a container with persistent volumes, limiting direct OS-level tuning.

Containerisation Constraints

- Services will be deployed using Docker Compose, which requires root access. This is possibly circumventable by using Podman as an intermediary.

Hosting constraints

- The system will run on a single server
- Resource usage must remain within the server limits (CPU, RAM, storage)
- No auto-scaling or distributed deployment is available

2.5.2 Security Constraints

- Passwords must be hashed using Argon2, requiring CPU-intensive hashing and secure configuration.
- We are using PyOTP for two-factor authentication, requiring time-synchronised OTP generation.
- All communication must occur over HTTPS.
- Strict input validation is required to prevent SQL injection, XSS, and CSRF.
- Role-based access control must be implemented for administrative features.
- Handling of payment information must be PCI-compliant.

2.5.3 Data Protection and Backup Constraints

- PostgreSQL data must be backed up using snapshot-based backups stored in external object storage.
- Backups must be stored off-server to avoid single-point-of-failure scenarios.

2.5.4 Organisational constraints

- ASC is a non-profit organisation with low revenue, requiring cost-efficient hosting and minimal recurring expenses.
- The system must be maintainable by volunteers or part-time administrators.
- Technology choices must prioritise long-term sustainability and low operational overhead.

2.5.5 Integration constraints

- The system will integrate with OJS for peer-review workflows, requiring compatibility with OJS metadata formats and APIs.
- Stripe will be used for payments, requiring secure API communication.
- Claude AI will be the default AI integration.

2.6 Assumptions and Dependencies

The design and operation of the ASC Knowledge Hub rely on several assumptions and external dependencies.

Note that technologies listed in this section are not final; this is our intended stack and our current assumptions, but elements may change during development.

2.6.1 System assumptions

- Users have access to a stable internet connection and a modern web browser.
- Administrators possess basic digital literacy and can manage content through the admin interface.
- ASC will maintain the accuracy and relevance of glossary terms, publications, and metadata.
- The hosting provider will maintain uptime, hardware reliability, and network connectivity.
- The system will not require horizontal scaling due to the ASC's modest traffic expectations.

2.6.2 Technical Assumptions

- We have sufficient access to the server to run Docker as root.
- PostgreSQL will remain stable and compatible with FastAPI ORM layers.
- Snapshot-based backups and object storage will remain accessible and affordable.
- External AI APIs (Claude) will remain available and stable.
- Nginx will handle TLS certificates and HTTPS termination reliably.

2.6.3 External dependencies

- Third-party services
    - Stripe must remain operational for payment processing.
    - Claude AI must remain available for AI-assisted features.
- Software dependencies
    - FastAPI, React, PostgreSQL, Docker, and related libraries must remain maintained.
    - OJS must remain maintained for peer-review integration.
    - Security libraries (argon2-cffi, PyOTP) must continue receiving updates.
    - Testing frameworks (pytest, RTL, Jest) must remain compatible with the codebase.
- Hosting dependencies
    - The server provider must supply CPU, RAM, and storage.
    - Server storage must remain available for snapshot-based backups.
    - SSH access must remain available for administrative maintenance.

---

3. Functional Requirements

3.1 Member Management

Member Application

- REQ-1.1 Non-members must be able to submit an application for membership.
- REQ-1.2 Non-members must be able to revise their application for membership.
- REQ-1.3 Managers must be able to approve, deny, or request revisions for an application.

Membership Status

- REQ-1.4 Users with an approved membership application must be able to start a membership by paying for it.
- REQ-1.5 Members must be able to cancel their membership, or change to a new membership type.
- REQ-1.6 Members must be able to opt-in to receive notifications (email & push-notification) on their membership status, such as when their membership is soon to expire.
- REQ-1.7 Members must be able to view their membership's payment plan.

Profile Management

- REQ-1.8 Members must be able to include fields on their profile for core profile information/identification, research interests, affiliations, publications and personal statements.
- REQ-1.9 Members must be able to link knowledge hub resources to be displayed on their profile.

Member Moderation

- REQ-1.10 Managers must be able to suspend or unsuspend a member.
- REQ-1.11 Managers must be able to import and export member profile information.
- REQ-1.12 Managers must be able to import and export membership payment information.

Extension Priority 10: Memberships Analytics Dashboard

- REQ-1.13 Managers must be able to view a member analytics dashboard for membership data, including renewal rates and pending renewal.

3.2 Groups

- REQ-2.1 Groups are automatically kept in sync with the ASC's google workspace mailing lists. The list of groups is 1-1 with mailing lists, and if someone is added or removed from a mailing list that should be reflected in their membership of the corresponding CKH group.
- REQ-2.2 Members are able to join or leave groups, which automatically adds or removes them from the appropriate mailing lists.
- REQ-2.3 Managers are able to assign or unassign users to groups.
- REQ-2.4 Managers are able to set permissions for groups, such that members are unable to join them — manager must assign users to these groups.

3.3 Knowledge Hub

Repository Management

- REQ-3.1 Administrators are able to create knowledge hub repositories, which have custom schemas. A schema will allow some very minimal customisation, semantically equivalent to the following json:

```
[
 {
  "label": "Heading",
  "type": "horizontal-block",
  "content": [
```

```
    {
      "label": "Title Image",
      "type": "image"
    },
    {
      "label": "Author",
      "type": "byline"
    }
  ]
 },
 {
  "label": "Content",
  "type": "text"
 },
 ...
]
```

- [Extension Priority 3: Administrator Visual Interface] REQ-3.2 Administrators are able to edit schemas through a visual interface.
- REQ-3.3 Administrators are able to configure repositories as "public" or "members only". This defaults to "members only".
- REQ-3.4 Managers are able to configure entries in repositories as "public" or "members only". This defaults to "members only".
- REQ-3.5 Administrators are able to give members management permissions for a repository.
- REQ-3.6 Managers are able to give members moderation permissions for a repository.

Content Submission

- REQ-3.7 Managers can configure repositories to bypass this approval process.
- REQ-3.8 Members are able to submit entries to repositories.
- REQ-3.9 Moderators are able to request revisions to a submitted entry pre-publication.
- REQ-3.10 Members are able to revise a submitted entry pre-publication.
- REQ-3.11 Moderators are able to approve a submitted entry.

Moderation

- REQ-3.11 Moderators are able to access the revision history of an entry.
- REQ-3.12 Moderators are able to unpublish an entry.

Extension Priority 6: Automated Summary

- REQ-3.13 Knowledge Hub repository schemas can include an "automated summary" section. For each entry, this is automatically populated with a summary of the rest of the text.
- REQ-3.14 The automated summary must be editable by a moderator.

3.4 Administration

- REQ-4.1 Administrators are able to enable/disable non-core modules.
- REQ-4.2 Administrators are able to set visibility permissions on non-core modules (public-facing, member-only, moderator-only, etc.).
- REQ-4.3 Administrators are able to export each module's data as an appropriate file format.
- REQ-4.4 Administrators are able to import a file to override a module's current data.
- REQ-4.5 Managers for a module are able to assign a member to be a moderator for that module.
- REQ-4.6 Administrators are able to assign a member to be a manager for a module.

N.B. per-module administration powers are included in that module's functional requirements.

3.5 Event Creation and Coordination

Extension Priority 7: Event Creation and Modification

- REQ-5.1 Managers must be able to create an event.
- REQ-5.2 Managers must be able to include fields for the name, date/time and venue.
- REQ-5.3 Managers must be able to delete an event.
- REQ-5.4 Managers must be able to update an event's details.
- REQ-5.5 Managers must be able to modify event branding (colour scheme, banner) to be displayed on the event's page for an event they have created.

[Depends on: Event Creation and Modification] Extension Priority 6: Event Engagement

- REQ-5.6 Members, or guests if the event is public-facing, must be able to subscribe (RSVP) to events.
- REQ-5.7 Members or guests must be able to unsubscribe from events.
- REQ-5.8 Members or guests must be able to receive reminders when a subscribed event is upcoming.
- REQ-5.9 Members or guests must be able to receive updates when a subscribed event is changed.
- REQ-5.10 Members or guests must be able to view a list of other members subscribed to an event.
- REQ-5.11 Members or guests must be able to sort events based on their fields/data.
- REQ-5.12 Managers must be able to give members moderation permission for a particular event.
- REQ-5.13 Moderators must be able to remove members or guests from events.
- REQ-5.14 Managers must be able to view an audit log of changes to an event.

[Depends on: Event Creation and Modification] Extension Priority 5: Event Content Submission

- REQ-5.15 Managers must be able to enable content submission for an event, which goes through the same content submission process as in 3.3 Knowledge Hub.
- REQ-5.16 Members or guests must be able to submit content to an event, which goes through the same content submission process as in 3.3 Knowledge Hub.
- REQ-5.17 Members or guests must be able to view an event's submitted content, after it has been published.

3.6 Knowledge Graph

- REQ-6.1 Members are able to view a simple knowledge graph visualisation that connects people and knowledge hub entries based on common tags and direct interaction.

Extension Priority 9: Advanced Knowledge Graph

- REQ-6.2 The knowledge graph algorithm must additionally analyse the relationship between people and knowledge hub entries as a two-mode network. See [1] for discussion of this type of data mining. Essentially, members interacting with the same knowledge hub entries should strengthen their connection, and similarly entries interacted with by the same member should have a stronger connection.

3.7 Member Matching

Smart member matching connects ASC members based on shared interests and skills. It processes information such as member profiles, publications and activity to determine relevant suggestions.

- REQ-7.1 Members must be able to receive a list of member match suggestions. Initially, this algorithm finds users with matching tags in their profiles.

N.B. See 3.10 Automated Metadata Suggestion for an important feature in improving the effectiveness of this feature.

- REQ-7.2 Members must be able to opt out of the matching system.
- REQ-7.3 Members must be able to dismiss a suggestion, in which case the system must not show that same suggestion for a period of 7 days (configurable in this module's settings).

Extension Priority 9: Advanced Member Matching

- REQ-7.4 The member matching algorithm must additionally analyse the relationship between people and knowledge hub entries as a two-mode network, and use this to match members. See [1] and Advanced Knowledge Graph for discussion on this technique.

Extension Priority 8: Adaptive Member Matching

- REQ-7.5 The system must log member feedback on suggestions (e.g. viewed or dismissed suggestion) and use this information to improve future matching.

3.8 Automated Newsletter

The automated newsletter feature automatically generates monthly summaries on platform activity, trends, members and recent publications.

- REQ-8.1 Every month (this duration is configurable in this module's settings) the system must automatically generate a newsletter, and submit it to a reserved "Newsletter" repository per the content submission process in 3.3 Knowledge Hub.
- REQ-8.2 The automated newsletter must consist of sections that managers can enable or disable in this module's settings. At first, these sections will be very basic facts and figures about the system. At minimum: a list of new members this month, the most popular tags on recent content and a list of upcoming events.
- REQ-8.3 Members must be subscribed to the "Newsletter" repository by default.
- REQ-8.4 Members must be able to unsubscribe from the "Newsletter" repository at any time.
- REQ-8.5 Managers must be able to manually cause the system to generate a newsletter at any time.

Extension Priority 6: Advanced Newsletter

- REQ-8.6 The system must include sections that identify more complex trends in member activity across the platform. At minimum: LLM-powered identification of trends in new knowledge hub entries and resurfacing old entries that have a recent uptick in engagement.

3.9 Research Harvester

This feature uses AI to search external online sources for relevant papers.

Extension Priority 8: Research Scraper

- REQ-9.1 Every day the system must scrape for relevant cybernetics-related papers from all supported sources.
- REQ-9.2 the system must support arXiv and JSTOR as sources for papers.
- REQ-9.3 Administrators can configure the scraping to happen at a different interval.
- REQ-9.4 Administrators can configure which paper sources are enabled.
- REQ-9.5 The system must put scraped papers into a dedicated "Scraped Papers" knowledge hub repository. By default, this bypasses the normal content submission approval process.

Extension Priority 4: Personalized Research Paper Feed

- REQ-9.6 The system must be able to provide a personalized feed of recent papers for each member.
- REQ-9.7 Members must be able to dismiss specific recommendations from their feed.
- REQ-9.8 The system must remember member feedback on recommendations for future discovery generation.

3.10 Automated Metadata Suggestion

Extension Priority 10

- REQ-10.1 When a member submits an entry to a Knowledge Hub repository, they are automatically provided with a list of tags. They can edit this automatically-generated list.
- REQ-10.2 When a user updates their profile, they are provided with a suggestion of how their tags should change. They can edit, approve or ignore this suggestion.

3.11 User Journeys

Creating Learning Journeys

- REQ-11.1 Administrators can configure a knowledge hub repository to be a learning journey repository. Entries in such a repository consist of a series of parts.
- REQ-11.2 While editing a learning journey Members must be able to reorder parts, insert new parts before or after existing parts, or delete a part.

Consuming Learning Journeys

- REQ-11.3 Members must be able to mark parts of a learning journey as complete. When all parts are complete, the entire journey is considered complete. Completion progress must persist across sessions.
- REQ-11.4 Members must be able to view a history of learning journeys they have interacted with — which they can filter by metadata such as whether they are still in-progress or have completed that journey.
- REQ-11.5 Members must be able to search for learning journeys based on how many users have completed the journey, on top of the usual search options for a knowledge hub repository.

3.12 Feedback

Rating

- REQ-12.1 Members must be able to upvote or downvote knowledge hub entries.

Comments

Members can respond to published knowledge hub entries; they are posted immediately without the need for approval. They rely on community feedback rather than active moderation.

- REQ-12.2 Members must be able to post comments on knowledge hub entries.
- REQ-12.3 Members must be able to upvote or downvote a comment.
- REQ-12.4 The system must collapse comments below a rating threshold, configurable in this module's settings.
- REQ-12.5 Members must be able to report a comment as harmful.
- REQ-12.6 The system must notify moderators if a comment has 2 or more reports.
- REQ-12.7 Moderators must be able to view all comments that were flagged as harmful.
- REQ-12.8 Moderators must be able to write, publish and edit a summary of comments on a knowledge hub entry.

Extension Priority 6: Automated Comments Summary

- REQ-12.9 The system must generate an AI summary of comments on a knowledge hub entry.
- REQ-12.10 Members must be able to delete their own comment, and moderators must be able to delete any comment.
- REQ-12.11 Members must be able to edit their own comment.

Bookmarks

- REQ-12.12 Members must be able to bookmark knowledge hub entries.

Read

- REQ-12.13 Members must be able to mark knowledge hub entries as read. (N.B. Some entries are links to external publications.)

---

4. External Interface Requirements

4.1 User Interfaces

The system must function as a responsive web app, accessible by web browsers without the need for native installation.

The system must provide consistent navigation for all user classes, with items shown or hidden based on Role Based Access Control.

The system must clearly display error messages, showing what went wrong and what action the user can take.

The system must support the latest two versions of Chrome, Safari and Dia for all supported platforms.

Extension Priority 3: Extended Platform Support

- The system must support the latest two versions of Chrome, Safari and Dia for all supported platforms.
- The system must be responsive, adapting layout to desktop, tablet and mobile screen sizes.

Extension Priority 2: Progressive Web Application

- The system must support installation to a device home screen.
- The system must be usable as a progressive web app.
- The system must cache relevant information, including at least: RSVPed events and bookmarked content.

- The system must allow users to download an entire knowledge hub repository (e.g. the glossary) for offline browsing.

4.2 Hardware Interfaces

The system must function on standard consumer desktop and laptop hardware.

Extension Priority 3: Extended Platform Support

- This extends to tablets and smartphones.
- The system must support touch-based navigation on mobile devices.

4.3 Software Interfaces

- The server must expose a REST API for communication between client and server.
- The system must integrate with the ASC's Google Workspace mailing lists.
- The system must NOT expose its database publicly, instead the back end will communicate with the database locally.
- The system must NOT directly handle payment information. It must instead use a payment processing provider to support any fees.
- The system must support secure authentication with an authenticator-based OTP.
- The system must support Argon2 for secure password hashing.
- The system must expose an authentication server that other ASC applications are able to reuse.

4.4 Communications Interfaces

- All client-server communication must exclusively occur over HTTPS.
- The system must communicate with external academic sources over secure APIs.
- The system must ensure HTTPS only communication with third party services, such as Anthropic's Claude and Stripe.

---

5. Other Nonfunctional Requirements

5.1 Performance

- The system must take no more than 2 seconds to display the main dashboard after a successful login with valid credentials, under normal operating conditions (>= 10 Mbps Connection).
- The system must display interface skeleton screens or loading indicators while content is being fetched, in accordance with Nielsen Norman Group response time guidelines, to maintain perceived performance during data loading.
- The system must not take more than 1.5 seconds to load pages when the user is navigating through different components of the app under a load of up to 100 concurrent users.
- The system must not take more than 2 seconds to return search results after a search request is performed by a user under a network speed of at least 10Mbps.
- The system must not take more than 5 seconds to render knowledge graphs containing up to 500 nodes upon user request.
- The app must not take more than 3 seconds to generate and display the profiles of researchers according to "smart member matching".
- The app must regenerate the smart member matching components within 30 seconds of the user updating their profile or research interests.

5.2 Safety

- The system must validate user inputs to protect the system from possible script attacks.
- The system database must automatically backup every 24 hours to prevent possible data loss.
- The system must maintain a full audit log and version history for all collaboratively edited resources, and the administrators must be able to reverse any accidental or malicious modifications or deletions.
- The system must display clear error messages during a system failure.
- The system's logging systems and error messages must NOT expose any sensitive data.
- Critical information of users including event fees and payment methods stored in the system must be encrypted to ensure safety of users.
- The administrators must have the ability to recover information and services within one hour after a critical system failure.

5.3 Security

- The system must enforce strong password policies requiring a minimum of 8 characters including uppercase and lowercase letters, numbers, and special symbols in accordance with OWASP recommendations.
- The system must use Argon2 password hashing algorithm combined with appropriate salting and cost factors to securely store user passwords.
- The system must support Multi-Factor Authentication for all member accounts.
- The system must implement JWT based authentication to manage user authentication sessions after a successful login.
- The system must enforce a temporary lock of 10 minutes on a user account if there are three consecutive failed login attempts. If three more wrong password attempts occur after the lockout ends, the account must be permanently locked until manually unlocked by an administrator.
- The system must implement Role-Based Access Control to restrict system access based on user roles, including member and administrator, ensuring users only have necessary permissions.
- The system must ensure all personal and profile data is protected from unauthorised access.
- The system must encrypt all sensitive data both in transit and at rest, using HTTPS/TLS.
- The system must maintain audit logs of critical actions including logins, account changes, and administrative activities.
- The system must protect against common web vulnerabilities including SQL Injection, Cross-Site Scripting and Cross-Site Request Forgery.
- The system must store authentication token only in HttpOnly cookies to prevent Cross Site Scripting attacks.
- The system must implement session expiry based on data sensitivity. Inactive sessions accessing sensitive data such as payment must expire after 15 minutes. Sessions accessing general content may remain active for up to 30 days on trusted devices. Users must be able to extend active sessions through token refresh while the session remains active.
- The system must provide a 'Remember Me' option in trusted devices, where users are able to maintain longer sessions in accordance with the above session expiry policy via token refresh.
- The system must ensure that automatically generated content does not expose any sensitive information — in particular, LLMs should not be given any information that would not be acceptable to include in its output.
- The system must comply with GDPR and CCPA/CPRA privacy regulations.

5.4 Software Quality

- The system must support at least 500 concurrent users without performance degradation.
- The platform must maintain a minimum of 99.5% uptime annually, excluding scheduling maintenance, ensuring that it is available for users 24/7.
- The system must ensure all content updates in the living cybernetic glossary are saved immediately and consistently.
- The system must implement automatic error logging and recovery mechanisms to minimise service disruptions.
- The system must provide an interactive and user-friendly interface that is suitable for users with varying levels of technical backgrounds.

- The system must use a scalable architecture capable of supporting large knowledge graphs and increasing user loads.
- The system must maintain a modular and testable codebase following standard software engineering practices.
- The system must provide well-documented APIs and developer documentation to enable support for future development or extension and maintenance.
- The system must log all AI and member activities, including edits and contributions, for required debugging and transparency.

Appendix A: Extension Priority

Priority for elements not in the MVP. (Higher priority is more important.)

Priority	Extensions
10	Automated Metadata Suggestion, Memberships Analytics Dashboard
9	Advanced Knowledge Graph, Advanced Member Matching
8	Adaptive Member Matching, Research Scraper
7	Event Creation and Modification, Advanced Newsletter
6	Event Engagement, Automated Content Summary, Automated Comments Summary
5	Event Content Submission
4	Personalized Research Paper Feed
3	Administrator Visual Interface, Extended Platform Support
2	Progressive Web Application

Appendix B: Handover Plan

- We will regularly sync our changes to an ASC-owned git repository.
- We will provide comprehensive documentation to support deployment and ongoing maintenance.
- We do NOT currently plan to handle setup of a VPS, or installation onto that VPS.

LLM Dependency

Some features depend on the use of an LLM model. We are going to support using Claude as a provider for this — though the system is designed to be extended to other providers. Use of these features will require configuring an API key.

Closer to the handover, we will compile a list of LLM-Dependent features, and their expected prices.

Payments Dependency

Some core features depend on a payment platform. We are going to support using Stripe as a provider for this — though the system is designed to be extended to other providers. Use of the system will require configuring an API key.

Recommend Environment

Our recommendation for hosting the ASC Knowledge Hub is to host it on a single VPS running Ubuntu 22.04 LTS, configured with SSH key authentication and basic firewall (UFW) for security. The system should be kept up to date to maintain stability and readability.

For the relatively low expected number of users, we are aiming for a system that can operate on 1 CPU, 2GB of memory and 50 GB of storage. Some examples of providers that would fit this requirement and offer USA-based hosting:

Provider	Specs	Price	Notes
OVH	4vCPU, 8 GB Memory, 75GB Storage, 400 Mbps	USD$7.60 per month	This is their cheapest option. Least flexibility with upgrades.
Vultr	1vCPU, 2GB Memory, 55GB Storage, Surcharges for >2TB bandwidth use.	USD$10 per month	Surcharges for >2TB bandwidth use. Docker integration. Customer service has been badmouthed online.
Microsoft Azure	2vCPU, 4GB Memory, 50 GB Storage, Charges for bandwidth	USD$25.2945 per month + ??? Bandwidth charges	"B2pls v2" instance + 50GB Standard SSD. Offers "Azure Grant" for nonprofits.

AWS Amazon Lightsail	2vCPU, 2GB Memory, 60GB Storage, Surcharges for >3TB bandwidth use.	USD$12 per month	Offers "AWS Nonprofit Credit Program"
InMotionHosting	4vCPU, 8 GB Memory, 160GB NVMe SSD, 5TB Bandwidth	$26.99/mo	Current host for ASC's website

Appendix C: UI

Page List — The Page List UI offers users various pages (Knowledge Hub submissions) that are grouped together by similar properties, such as by tags. *[Wireframe shows a sidebar layout with two rows of card-style content blocks and right-arrow navigation.]*

Page — The Page UI offers users a page's content, its identifying features (such as title and author), and buttons with actions to interact with the page such as bookmarking it. *[Wireframe shows a sidebar layout, a centered image with action buttons below it, and a paragraph block.]*

Event List — The Event List UI offers users a list of events, with the images and their essential data visible in a card format down the screen. *[Wireframe shows a sidebar layout with two large card blocks stacked vertically.]*

Event — The Event UI offers users a detailed page for an event, with its associated identifying data, image, themes applied to the webpage, event information, and actions to interact with the event. *[Wireframe shows sidebar layout, large hero block at top with action buttons, and content paragraph below.]*

Groups List — The Groups List UI offers users a list of available groups, including quick actions to join them, and some associated identifying data. *[Wireframe shows sidebar layout with four card blocks in a row, each with action buttons.]*

Group — The Group UI offers users a detailed view of a group and its members, actions to interact with the group, and subgroups of the selected group if they exist. *[Wireframe shows sidebar layout with header block, three subgroup cards with a right-arrow, and a right-side member list.]*

Knowledge Graph — The Knowledge Graph UI offers the ability to enact queries on the Knowledge Hub, filter the Knowledge Hub, or otherwise view/interact with a dynamic knowledge graph. *[Wireframe shows sidebar layout with a large central content area containing a globe icon.]*

Search — The Search UI offers a page with multiple types of Knowledge Hub data/features, such as pages (Knowledge Hub submissions), events, groups, and other related data, allowing the user to search and find any relevant content. *[Wireframe shows sidebar layout, search bar at top, a row of small cards, and two larger card blocks below.]*

Appendix D: References

[1] Krebs, V. (2010). *Beautiful Visualization* Chapter 7 (J. Steele & N. Iliinsky, Eds.). Shroff - O'reilly. Retrieved May 17, 2026, from http://www.orgnet.com/Beautiful_Visualization_Krebs_Chapter7.pdf